25

Using Triggers

In this chapter, you'll learn what triggers are and why and how they are used. You'll also learn the syntax for creating and using them.

Understanding Triggers

> **Note**
> **This Chapter Requires MySQL 5** Support For Triggers Was Added To MySQL 5. Therefore, This Chapter Is Applicable To MySQL 5 Or Later Only.

MySQL statements are executed when needed, as are stored procedures. But what if you want a statement (or statements) to be executed automatically when events occur? For example:

- Every time a customer is added to a database table, you want to check that the phone number is formatted correctly and that the state abbreviation is in uppercase.
- Every time a product is ordered, you want to subtract the ordered quantity from the number in stock.
- Whenever a row is deleted, you want to save a copy in an archive table.

What all these examples have in common is that they need to be processed automatically whenever a table change occurs. And that is exactly what triggers do. A *trigger* is a MySQL statement (or a group of statements enclosed within `BEGIN` and `END` statements) that is automatically executed by MySQL in response to any of these statements:

- `DELETE`
- `INSERT`
- `UPDATE`

No other MySQL statements support triggers.

Creating Triggers

When creating a trigger, you need to specify four pieces of information:

- The unique trigger name
- The table with which the trigger is to be associated
- The action that the trigger should respond to (`DELETE`, `INSERT`, or `UPDATE`)
- When the trigger should be executed (before or after processing)

Tip

Keep Trigger Names Unique per Database Unlike most DBMSs, MySQL requires trigger names to be unique per table but not per database. This means that two tables in the same database can have triggers of the same name.

Triggers are created using the `CREATE TRIGGER` statement. Here is a really simple (and not overly useful) example:

Input

```
CREATE TRIGGER newproduct AFTER INSERT ON products
FOR EACH ROW SET @result = 1;
```

Analysis

Here `CREATE TRIGGER` is used to create the new trigger named `newproduct`.

Triggers can be executed before or after an operation occurs, and here `AFTER INSERT` is specified so the trigger will execute after a successful `INSERT` statement has been executed. The trigger then specifies `FOR EACH ROW` and the code to be executed for each inserted row. In this example, a variable named `@result` will be set to `1`.

To test this trigger, use the `INSERT` statement to add one or more rows to `products`. You can then use `SELECT` to display the variable.

Note

Only Tables Triggers are only supported on tables, not on views. (They are also not supported on temporary tables.)

Triggers are defined per time per event per table, and only one trigger per time per event per table is allowed. Up to six triggers are supported per table (before and after `INSERT`, `UPDATE`, and `DELETE`). A single trigger cannot be associated with multiple events or multiple tables, so if you need triggers to be executed for both `INSERT` and `UPDATE` operations, you'll need to define two triggers.

Note
When Triggers Fail When a `BEFORE` trigger fails, MySQL does not perform the requested operation. In addition, when either a `BEFORE` trigger or the statement itself fails, MySQL does not execute an `AFTER` trigger (if one exists).

Dropping Triggers

By now the syntax for dropping a trigger should be apparent. To drop a trigger, you use the `DROP TRIGGER` statement, as shown here:

► Input

```
DROP TRIGGER newproduct;
```

► Analysis

Triggers cannot be updated or overwritten. To modify a trigger, you must drop it and re-create it.

Using Triggers

Now that we've covered the basics of triggers, we will now look at each of the supported trigger types and the differences between them.

INSERT Triggers

An `INSERT` trigger is executed before or after an `INSERT` statement is executed. Be aware of the following:

- In `INSERT` trigger code, you can refer to a virtual table named `NEW` to access the rows being inserted.
- In a `BEFORE INSERT` trigger, the values in `NEW` may also be updated (so you can change values that are about to be inserted).
- For `AUTO_INCREMENT` columns, `NEW` contains `0` before the trigger and the new automatically generated value after the trigger.

Let's look at an example—a really useful one. `vendors` contains a column named `vend_state` that stores a vendor's state (as part of its address). Ideally state abbreviations should be capitalized, such as `CA` rather than `ca` or `Ca`. You could ask users to always enter the data correctly, but, yeah right! Using a trigger is a better solution:

► Input

```
DELIMITER //

CREATE TRIGGER newvendor AFTER INSERT ON vendors
FOR EACH ROW
```

```
BEGIN
   UPDATE vendors SET vend_state=Upper(vend_state) WHERE vend_id = NEW.vend_id;
END //

DELIMITER ;
```

Analysis

This code creates a trigger named `newvendor` that is executed by `AFTER INSERT ON vendors`. When a new vendor is saved in `vendors`, a copy is saved in `NEW`, and `NEW.vend_id` contains the ID of the newly inserted vendor. The trigger contains an `UPDATE` statement that updates `vend_state` with `Upper(vend_state)` and uses `NEW.vend_id` in the `WHERE` clause so that the right row is updated. Now, no matter what the user enters, the data will be stored correctly.

To test this trigger, add a new vendor and then use `SELECT` to verify that `vend_state` was indeed updated.

Tip

BEFORE or AFTER? As a rule, you should use `BEFORE` for any data validation and cleanup to ensure that the data inserted into the table is exactly as needed. This applies to `UPDATE` triggers, too.

DELETE Triggers

A `DELETE` trigger is executed before or after a `DELETE` statement is executed. Be aware of the following:

- Within `DELETE` trigger code, you can refer to a virtual table named `OLD` to access the rows being deleted.
- The values in `OLD` are all read-only and cannot be updated.

The following example demonstrates the use of `OLD` to save rows that are about to be deleted into an archive table:

Input

```
DELIMITER //

CREATE TRIGGER deleteorder BEFORE DELETE ON orders
FOR EACH ROW
BEGIN
   INSERT INTO archive_orders(order_num,
                              order_date,
                              cust_id)
   VALUES(OLD.order_num,
          OLD.order_date,
          OLD.cust_id);
```

```
END//

DELIMITER ;
```

► Analysis

Before any order is deleted, this trigger is executed. It uses an `INSERT` statement to save the values in `OLD` (the order about to be deleted) into an archive table named `archive_orders`. (To actually use this example, you need to create a table named `archive_orders` with the same columns as `orders`.)

The advantage of using a `BEFORE DELETE` trigger (as opposed to an `AFTER DELETE` trigger) is that if, for some reason, the order cannot be archived, the `DELETE` will be aborted.

> **Note**
> **Multi-Statement Triggers** Notice that the trigger `deleteorder` uses `BEGIN` and `END` statements to mark the trigger body. This is actually not necessary in this example, although it does no harm being there. The advantage of using a `BEGIN END` block is that the trigger can then accommodate multiple SQL statements (one after the other within the `BEGIN END` block).

UPDATE Triggers

An `UPDATE` trigger is executed before or after an `UPDATE` statement is executed. Be aware of the following:

- Within `UPDATE` trigger code, you can refer to a virtual table named `OLD` to access the previous (pre-`UPDATE` statement) values and `NEW` to access the new updated values.
- In a `BEFORE UPDATE` trigger, the values in `NEW` may also be updated so that you can change the values that are about to be used in the `UPDATE` statement.
- The values in `OLD` are all read-only and cannot be updated.

The following example revisits the `INSERT` example used previously and ensures that state abbreviations are always in uppercase, even when updated:

► Input

```
DELIMITER //

CREATE TRIGGER updatevendor BEFORE UPDATE ON vendors
FOR EACH ROW
SET NEW.vend_state = Upper(NEW.vend_state) //

DELIMITER ;
```

► Analysis

This version works a little differently than the earlier one. Rather than save the row and then update it, this version does data cleanup before using `BEFORE UPDATE`. Each time a row is updated, the value in `NEW.vend_state` (the value that will be used to update table rows) is replaced with `Upper(NEW.vend_state)`.

More on Triggers

Before wrapping up this chapter, here are some important points to keep in mind when using triggers:

- Creating triggers might require special security access. However, trigger execution is automatic. If an `INSERT`, `UPDATE`, or `DELETE` statement is executed, any associated triggers are executed, too.
- Triggers should be used to ensure data consistency (case, formatting, and so on). The advantage of performing this type of processing in a trigger is that it always happens, and it happens transparently, regardless of the client application.
- One very interesting use for triggers is to create an audit trail. By using triggers, it would be very easy to log changes (even before and after states, if needed) to another table.
- Unfortunately, the `CALL` statement is not supported in MySQL triggers. This means that stored procedures cannot be invoked from within triggers. Any needed stored procedure code needs to be replicated within the trigger itself.

Summary

In this chapter, you learned what triggers are and why they are used. You learned about the trigger types and when they can be executed. You also saw examples of triggers used for `INSERT`, `DELETE`, and `UPDATE` operations.